CATHOLIC UNIVERSITY OF EASTERN AFRICA



FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

UNIT

CMT 311: Fundamentals of Software Engineering

PROJECT TITLE

AgriLink Mobile Based Agricultural Marketplace

GROUP MEMBERS

Hailu Dabaly - 1062554
Upao Mazibo - 1062548
Daniel Mbithi - 1060901

Project repository: https://github.com/dabaly/AgriLink.git

Project overview

AgriLink is a mobile-friendly website that helps farmers publish agricultural goods and helps buyers find and purchase them. It includes verified accounts, listings, buyer-farmer conversations and offers, orders, payment processing, delivery or pickup, reviews, disputes, notifications, and administrator tools.

The application is written in Python using Flask. It stores its information in a relational database, presents pages through a web browser, and uses a real-time connection for some chat and notification updates. The project can be run locally with SQLite and mock providers. Real SMS and payment services need deployment credentials and provider-side setup.

Planning and Feasibility Analysis

Problem and goals

Farmers need a practical way to display goods and talk with buyers. Buyers need to search for goods, agree on price and quantity, pay, and arrange pickup or delivery. The platform team needs ways to moderate listings, manage disputes, and review administrative actions.

- Help farmers publish and manage agricultural listings.
- Help buyers find goods, communicate with farmers, and complete purchases.
- Keep accounts, payments, orders, and delivery details available only to the people allowed to see them.
- Give administrators controlled tools for platform support and moderation.

People involved

Group	What they need
Farmers	Publish accurate listings, manage stock, respond to buyers, and fulfill orders.
Buyers	Find goods, negotiate, pay, receive purchases, and get help with problems.
Administrators	Manage account status, moderate listings, review disputes, and inspect audit records.
Development and operations team	Build, test, deploy, secure, monitor, back up, and support the service.
Service providers	Send phone verification messages and process or confirm payments.

Feasibility Study

Feasibility area	AgriLink assessment
Technical	The project uses established Python web tools. The production database, hosting, file storage, and real-time messaging setup still need environment-specific validation.
Economic	No project budget is recorded. Estimate hosting, database and image storage, SMS, payment fees, monitoring, backups, and support.
Operational	Main user and administrator workflows exist. Scheduled jobs, central monitoring, recovery procedures, and provider operations must be configured by deployment staff.
Legal and privacy	The system holds phone numbers, delivery contacts, and transaction records. The deployment owner must define applicable privacy, retention, and deletion rules.
Schedule and people	No staffing plan or project calendar is included. Each feature estimate should include design, coding, checks, deployment, and support.

Risks Analysis

Risk	Control
Someone abuses or takes over an account	Hashed passwords, limited-use phone codes, attempt and resend limits, and rate limiting.
A user sees another person's private order or chat	Check the person's role and relationship to each record in routes and business services.
Two buyers claim the same stock	Check and update stock together in one database transaction when an offer is accepted.
A fake or repeated payment message changes an order	Verify provider messages, record unique event IDs, and process each event safely once.
Private information or secrets appear publicly	Return only approved public fields and keep credentials and sensitive data out of logs.
An upgrade or deployment fails	Review database changes, create a backup, and rehearse restoration before production changes.

Requirements Analysis and Specification

Requirements state what the system must do and how well it must do it. The points below summarize the behavior present in the project.

Functional Requirements

- A person can register with a phone number and password, verify the number with a one-time code, and choose Farmer or Buyer. Administrators are created through a protected terminal command.
- Visitors can browse, search, filter, sort, and page through public listings and seller summaries.
- Verified farmers can create and manage listings, including allowed listing photos. Only active, approved, available listings with stock appear publicly.
- Verified buyers can contact a listing farmer. Both participants can send messages and make or respond to offers.
- Accepting an eligible offer checks and reduces available stock and creates an order that records the agreed product, quantity, and price.
- Buyers and farmers can view orders they participate in and complete the actions permitted at each order stage. Pickup or delivery information is collected as required.
- A buyer can start a payment for a confirmed order. The server chooses the amount; a trusted provider result is required before an order is marked paid.
- Eligible buyers can review completed orders. A buyer or farmer involved in an eligible order can open a dispute for administrator review.
- Users receive in-app notifications about relevant activity. Administrators can manage account status, listings, disputes, and audit records.

Non Functional Requirements

Requirement	Description
Security	Only the right people can use protected actions; passwords and provider secrets are protected; forms resist unwanted requests.
Privacy	Public pages do not reveal private phone numbers, email addresses, delivery details, or payment information.
Accuracy	Prices, stock, payments, and order states must remain consistent even when actions are repeated or happen at the same time.
Reliability	Important transactions and messages are saved. A provider retry should not accidentally charge or update an order twice.

Requirement	Description
Usability	Farmers, buyers, and administrators can understand and complete their common tasks on mobile and desktop screens.
Maintainability	Features are organized by business area so developers can fix or extend one part without rewriting the whole application.

Acceptance Guide Rules

- Prices are stored as whole-number KES cents, and the server calculates the amount to charge.
- Phone verification codes expire and have attempt and resend limits. The mock SMS service is for development only.
- Stock is reduced when an offer is accepted. The order stores its own purchase snapshot, so fulfillment does not reduce stock a second time.
- A cancellation may return stock before the goods leave the seller. Stock is not restored after physical handoff.
- Only authorized people can change orders, payments, reviews, disputes, and listings; important changes must leave a history record.
- Payment notifications must be verified and safe to retry.
- Passwords, verification codes, secrets, raw provider messages, and private contact details must not appear in public output or logs.

System and Software Design

Design is the plan that shows how the parts of the application fit together. AgriLink works like a service desk: the web page receives a request, the business service checks the rules, and the database records the result. Separate adapters handle outside services such as SMS and payment providers.

Application Requests

A buyer or farmer uses a browser, a JSON client, or a live chat connection. A Flask route receives the request. A domain service applies the marketplace or transaction rules. The service reads or updates database models and may call a provider adapter. The page or response is then returned to the user.

Part	Description	Project location
Routes and forms	The front counter: receives requests, checks submitted information, and chooses the page or response.	`app/*/routes.py`, `app/*/forms.py`
Domain services	The rulebook: decides whether an action is allowed and saves related changes together.	`app/*/services.py`
Models and database	The filing system: stores users, listings, messages, offers, orders, payments, and histories.	`app/models/`, `migrations/`
Provider adapters	Translators between AgriLink and outside SMS or payment services.	`app/integrations/`, `app/payments/providers.py`
Templates and static files	The screens and browser behavior people see.	`app/templates/`, `app/static/`
Socket.IO handlers	The live connection used for chat and notification updates.	`app/chat/sockets.py`

Design Choices

- Each business area has its own package, such as marketplace, orders, payments, or notifications. This makes the code easier to find and change.
- Business rules belong in services. A route should not be the only place checking who owns an order or whether stock is available.

- The database uses constraints and indexes to reject invalid records and speed up common searches.
- Database migrations are versioned instructions for changing the database structure. They let the team apply changes in a known order.
- Uploaded photos are resized and stored outside the public website folder. The application checks access before showing them.
- A payment return page does not prove that money was received. The application waits for a verified provider message or status check.
- The default local database is SQLite. Production database and realtime hosting choices need testing in the real deployment environment.

Database Design

AgriLink uses SQLAlchemy 2 models, with Flask-Migrate (Alembic) for versioned changes. There are 9 migrations and 21 tables, grouped by business area:

Area	Tables
Accounts	users, profiles, otp_challenges, otp_request_events
Marketplace	categories, listings, listing_images
Negotiation	conversations, messages, offers
Orders	orders, order_items, order_status_history, deliveries
Payments	payments, payment_events
Feedback	reviews, disputes, dispute_history
Platform	notifications, audit_logs

The database enforces key rules itself, so a coding mistake cannot silently break them:

- **Allowed values:** status, role, unit, grade, and currency fields use `CHECK` constraints. Prices, quantities, and totals must be positive.
- **One of each:** one review and one dispute per order, one order per accepted offer, one conversation per listing and buyer.
- **Partial unique indexes:** only one pending offer per conversation, one primary photo per listing, one active payment attempt and one successful payment per order.
- **Append-only records:** order history, order item snapshots, and audit logs reject updates and deletes.
- **Safe deletion:** foreign keys use `RESTRICT` for money and history records, so they cannot be removed by deleting a user or listing.
- **Search speed:** composite indexes support public listing filters (status, category, county, price) and newest-first lists for messages, notifications, and seller history.
- **Money:** amounts are whole-number KES cents (`Integer`), never floating point.
- **Time:** timestamps are stored timezone-aware in UTC and shown in Nairobi time.

SQLite is the local default, with foreign keys, write-ahead logging, and a busy timeout switched on at startup. The indexes also declare PostgreSQL variants, but migrations have only been tested on SQLite.

Implementation and Coding

Implementation means turning the design into working code. A contributor first finds the part of the application that owns the change, then updates its rules, screen, stored data, and documentation as needed.

Technology Stack

Layer	Technology
Language	Python 3.14 or newer
Web framework	Flask 3.1 with blueprints (one per business area)
Database access	Flask-SQLAlchemy / SQLAlchemy 2, Flask-Migrate (Alembic)
Accounts and security	Flask-Login (sessions), Flask-WTF (forms and CSRF protection), Flask-Limiter (rate limits), Werkzeug password hashing
Real-time	Flask-SocketIO with simple-websocket
Images	Pillow (verify, rotate, resize, strip metadata)
Phone and external calls	phonenumbers (Kenyan number handling), requests (SMS and payment providers)
Providers	Africa's Talking (SMS), M-Pesa Daraja, and Stripe, plus mock versions for development
Front end	Jinja templates, Bootstrap 5.3 and Socket.IO client loaded from a CDN, one small app.js
Quality tools	pytest and pytest-flask, Ruff, Bandit
Configuration	Environment variables through python-dotenv; development, testing, and production profiles in app/config.py

Project Setup

Python 3.14 and Git (for cloning the repository). The following commands install the project, prepare the local database, load the built-in listing categories, and start the development server:

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements-dev.txt
cp .env.example .env
flask --app wsgi.py db upgrade
flask --app wsgi.py seed
flask --app wsgi.py run
```

The development database and uploads live under `instance/`. The mock OTP provider writes development codes locally. Real passwords, provider keys, or production data are not stored in local files or source control.

Code Review and Version Control

Ruff checks code style and common mistakes; Bandit checks for selected security issues. The project tracks source files, tests, and database migrations in Git. Local settings, caches, logs, and database files are excluded. No formal branch policy, code review process, release tag policy, or automated build pipeline is documented in the repository.

Testing and Quality assurance

Automated checks

The test suite uses pytest. Shared test setup is in `tests/conftest.py`; test files cover application setup, accounts, marketplace, chat, orders, payments, notifications, feedback, and administrator features. Run these checks before a release:

```
pytest
ruff check .
bandit -r app
```

Functionality Checklist

- Test the normal path, invalid information, unauthorized access, and repeated requests.
- Check that a buyer cannot read another buyer's order and that a non-admin cannot use admin tools.
- Check that stock, totals, payment state, and order history stay correct when an action fails or is retried.
- Check that invalid provider callbacks are rejected and valid repeated events are not processed twice.
- Check that pages and API responses do not reveal private account, delivery, or payment details.
- Check photos with invalid file types, excessive size, or damaged image data.
- For a database change, apply the migrations to a clean database and a copy of a database at the supported prior version.
- Ask representative users to complete important tasks on phone and desktop screen sizes.

Tests use an in-memory SQLite database. The repository does not contain a CI pipeline, so the team must run and review the checks as part of its release process. Real payment testing requires provider sandbox credentials and careful coordination.

Deployment

Deployment means putting a tested version where real users can access it. It includes configuring the server, database, files, security settings, and outside service callbacks. The repository README explains application setup.

Production Configuration

- Set `AGRI_LINK_CONFIG=production`.
- Create a unique, strong `SECRET_KEY` and a separate `OTP_PEPPER`.
- Set `DATABASE_URL` to the production database and provide provider credentials through a protected environment or secrets manager.
- Use HTTPS so login cookies and user information travel securely.
- Configure persistent storage for listing photos and a production-compatible setup for Socket.IO if live chat updates are enabled.
- Do not use mock OTP or mock payment callbacks in production.

- Configure provider callback URLs and verify them in a sandbox before accepting real payments. Live M-Pesa and Stripe transactions have not been tested by this project.

Release steps

1. Choose and review the code version to release; run the automated checks.
2. Confirm secrets, database access, HTTPS, photo storage, logging, and provider callback settings.
3. Create a database backup and confirm that the team can restore it.
4. Apply database updates with `flask --app wsgi.py db upgrade`.
5. Load marketplace categories with `flask --app wsgi.py seed` if needed; it is safe to repeat.
6. Start the application with a server that supports the selected Flask and Socket.IO setup.
7. Check that the home page, login, public listings, database pages, photos, and provider callbacks work.
8. Watch application and provider records after release and record what version was deployed.

Administrator setup

Create an administrator through a trusted terminal. Public registration cannot create administrator accounts:

flask --app wsgi.py create-admin

Maintenance and evolution

Maintenance type	Definition	AgriLink application
Corrective	Fix something that is broken.	Repair an order screen that fails to show the correct status.
Adaptive	Adjust to a change outside the application.	Update an SMS or payment adapter after a provider changes its service.
Perfective	Improve a feature users already have or add useful behavior.	Make listing search easier or improve the mobile ordering experience.
Preventive	Reduce the chance of future problems.	Update dependencies, strengthen tests, rehearse database migrations, and improve monitoring.

Routine Maintenance

AgriLink includes command-line jobs for work that should happen after a period of time. A scheduler such as cron or the hosting platform's scheduler must run these commands regularly:

flask --app wsgi.py jobs run-maintenance
flask --app wsgi.py jobs expire-offers
flask --app wsgi.py jobs complete-orders

The combined maintenance command expires offers that passed their response deadline and completes delivered orders after the buyer review period. Operators should monitor whether the scheduled command finishes successfully.

Monitoring, recovery, and data care

- Collect application and web-server logs so operators can find errors. Do not log passwords, one-time codes, provider secrets, or raw payment callbacks.

- When investigating a payment or order problem, compare the application history with the provider's records before correcting anything.
- Back up the database and listing photos together, restrict access to backups, and test that the backup can be restored.
- SQLite uses a write-ahead log. Use a SQLite-aware backup method while the application is running; copying only the main file can miss recent changes.
- Set retention and account-data deletion rules for the deployment. The project does not currently provide a general account deletion flow.
- When retiring the service, stop new activity safely, keep records required for transactions or audits, remove provider credentials and callbacks, verify a final backup, then dispose of data under the retention rules.